

Locality and Fast File System

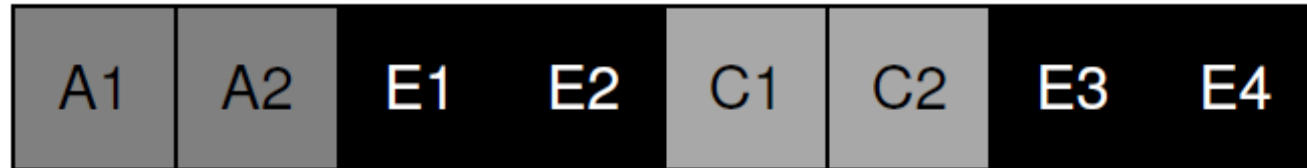
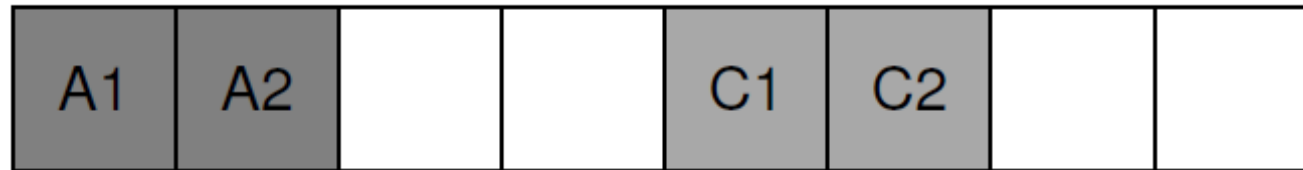
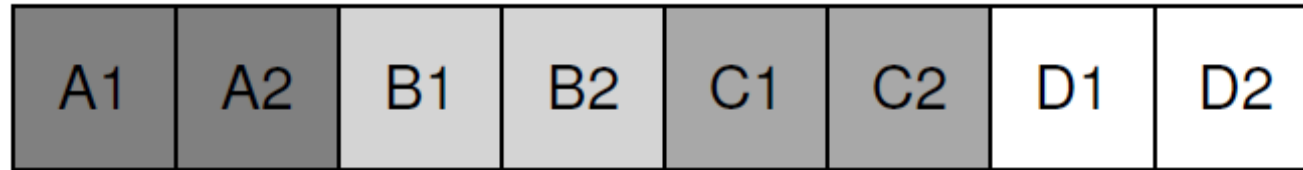
Chapter 41 - OSTEP

Original Unix File System



- Written by Ken Thomson. Very simple.
- Superblock (S) contains:
 - Volume Sizes
 - How many inodes.
 - Pointer to head of free list of data blocks.
- Problems. Performance becomes terrible with time:
 - Treated disk as Random Access, spreading data all over the disk: file data blocks far away from inodes
 - Disk fragmentation with time: logically contiguous file is spread all over the place incurring seeks
 - Small block size (512 bytes): positioning overhead

Old Unix FS Problems: Fragmentation

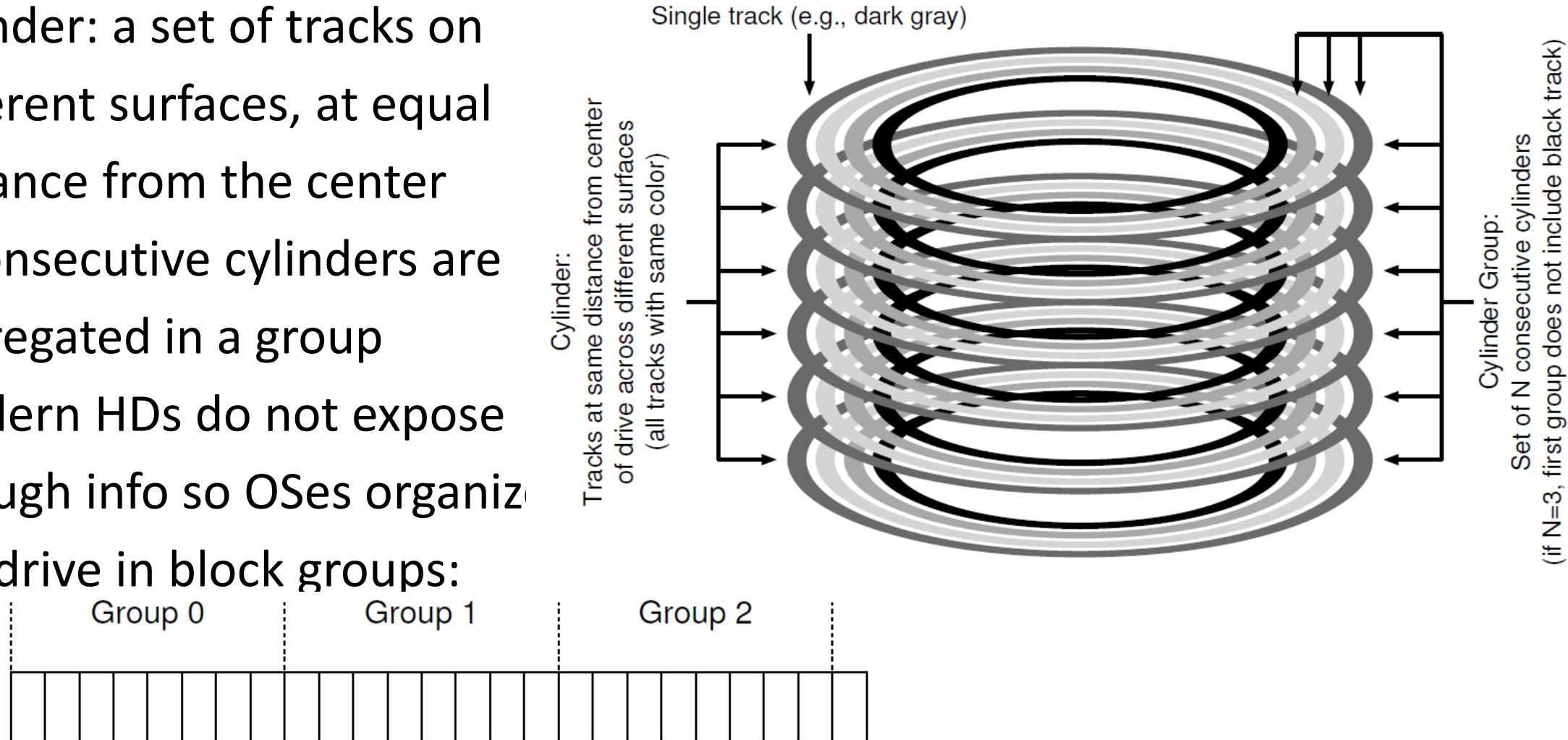


Fast File System

- Disk awareness is the solution.
- Kept the same interface: `open()` , `read()` , `write()`
- Changed the implementation to be disk aware.
- Changed the on-disk data structures:

Cylinder group

- Cylinder: a set of tracks on different surfaces, at equal distance from the center
- N consecutive cylinders are aggregated in a group
- Modern HDs do not expose enough info so OSes organize the drive in block groups:



Cylinder groups

- The idea is that two files placed in the same group will low seek overhead when accessed consecutively.
- To achieve the ability to put files and directories in the same group, the OS puts all the required structures in each cylinder group:



- The basic idea is keep related stuff together (unrelated stuff apart)
- Heuristics for a directory:
 - Find an group with low no. of directories (to balance dirs. across groups) and a high number of free inodes (to allocate subsequent files)
 - Put the inode and directory data in that group.

Cylinder groups contd.

- Heuristic for file:
 - Allocate the data blocks of the file in the same group as its inode
 - Put all files in the same directory in the group of that directory
 - Example:
 - Directories (/, /a, /b)
 - Files (/a/c, /a/d, /a/e, /b/f)
 - Dir size: 1 block
 - File size: 2 blocks

- FFS allocation →
- Improves locality. E.g. compiling a
Reads many files, create exe in same dir

group	inodes	data
0	/-----	/-----
1	acde-----	accddee---
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
...		

Does locality exists in FS accesses?

- SEER benchmark traces vs Random files
- X-axis: how far up the path is the common ancestor
- Y-axis: cumulative frequency of access
- Observation for SEER accesses:
 - 7% of accesses to same file
 - ~40% to same file + files in same directory (1 up)
 - 25% to files with same grand parent
(`/proj/src/file.c` , `/proj/include/file.h`)

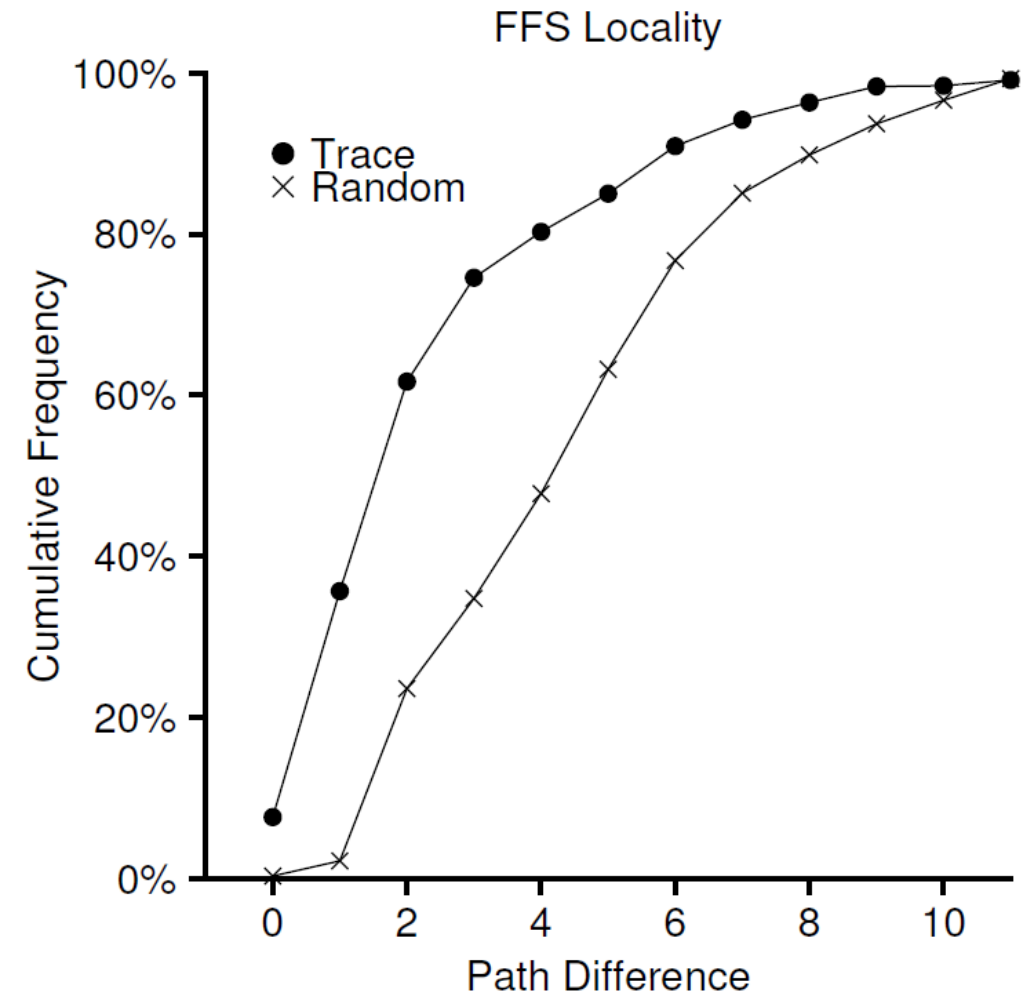


Figure 41.1: FFS Locality For SEER Traces

Large File Exception

- A large file will eat up most data blocks, rendering inodes useless

group	inodes	data
0	/a-----	/aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a-----
1	-----	-----
2	-----	-----
...		

- Spreading it across groups makes sure a single group utilization is not too high

group	inodes	data
0	/a-----	/aaaaa-----
1	-----	aaaaa-----
2	-----	aaaaa-----
3	-----	aaaaa-----
4	-----	aaaaa-----
5	-----	aaaaa-----
6	-----	-----
...		

Large File Exception contd.

- Spreading a file across groups incurs seek overhead reducing HD bandwidth
- This can be amortized by a large chunk size
(tradeoff: group utilization/locality vs bandwidth)
- FFS:
 - 12 direct blocs in same group as inode
 - Each indirect blocks and blocks pointed to in same group

